K215-R6

Two checked 215-state machines from row-local parameter permutations

Research collaboration for Vladimir Reshetnikov's ProveIt project

21 October 2026

Abstract

We construct two concrete binary, one-tape Turing machines with 215 working states. Each differs from its accepted K215 parent in exactly two transition rows and changes the sixth axiom generator from $\mathsf{Ex}_A(Q_A C) \to C$ to $\mathsf{Ex}_A(Q_A B) \to B$. The complete numerical axiom family is unchanged, including all undecodable natural-number codes. A repaired axiom-selection compiler, the inherited native calculus calibration, exact blank initialization, and a fresh quantified primitive-runtime check give an ordinary proof, formalizable in ZFC, that blank nonhalting is equivalent to Con(ZFC).

Both machines pass all 6,176 guarded runtime cases. Their exact blank prefixes take 95,714 and 96,284 transitions. A separate auditor checks source changes, case records, positive clocks, every transition row, state renaming, and the entire finite startup trace without importing the runtime checker. Neither machine is a state renaming of the four accepted K215 presentations included for comparison, nor of the other new machine.

No 214-state machine is claimed. The ρ_6 permutation itself has prior Source163 provenance. The contribution is its exact two-row composition with both accepted K215 command orders, complete fresh runtime verification, explicit source/calculus proof, and a portable reproduction packet. Additional results include a bounded exhaustive census of 279,936 row recipes, controlled allocation experiments, and a 215-state command permutation that passes primitive checks but halts from blank. No new proof-assistant endpoint or unconditional consistency assertion is claimed.

Deliverable at a glance

	`K215-R6-210`	`K215-R6-120`
Working states / transition rows	215 / 430	215 / 430
Source primitive occurrences	4,751	4,751
Guarded runtime cases proved	6,176	6,176
Symbolic proof-rule applications	1,699,838	1,699,893
Smallest duration constant	6	9
Blank startup transitions	95,714	96,284

The recommended integration candidate is `K215-R6-120`: it combines the sixth-row permutation with the accepted DROP0/NIP1/MP2 command order. Both complete tables, sources, reports and certificates are supplied.

Contents

1 Claim, conventions and provenance 3

2 The exact constructions 3

3 A numerical calculus that tolerates junk codes 5

4 Command semantics and soundness 6

5 A repaired completeness compiler 7

6 Whole-tape binary verification 8

7 Exact blank initialization and proof composition 10

8 Material difference and its precise extent 10

9 Experiments toward 214 11

10 Reproduction, review and integration 13

A Exact candidate identities 15

1 Claim, conventions and provenance

We use the usual deterministic two-symbol, one-tape convention. Tape positions are integers, the blank symbol is 0, the initial head is at position 0, and each transition writes one bit and moves exactly one cell left or right. A distinguished halt label is not a working state. Entering that label halts. There are two defined rows for every working state.

In the native presentations, start is 0, dispatcher is 84 and halt is 254. The dense presentations relabel the sorted working-state set to $0, \ldots, 214$ and relabel halt to 215; their dispatcher is 83. Every one of the 215 working states is reachable in the directed transition graph from start. This graph statement does not assert that all states occur on the actual blank computation.

Theorem 1.1 (Ordinary mathematical endpoint)**.** *For each of the two exact machines M supplied in this packet, the following equivalences have a proof formalizable in ZFC:*

$$\mathsf{ZFC} \vdash \mathrm{Halt}(M, \mathrm{blank}) \iff \neg\,\mathrm{Con}(\mathsf{ZFC}), \qquad \mathsf{ZFC} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \iff \mathrm{Con}(\mathsf{ZFC}).$$

Here $\mathrm{Con}(\mathsf{ZFC})$ *means the usual syntactic consistency assertion for ZFC, with the proof coding connected to the inherited native-calculus calibration.*

The proof uses the repository's accepted numerical-calculus calibration and dominated-ρ_8 recovery theorem as predecessor results [1, 4, 3]. This packet supplies the changed source-to-calculus argument and new exact binary checks. It does not contain a newly compiled Lean or Rocq theorem for these two exact tables, an internal-PA endpoint, a proof of $\mathrm{Con}(\mathsf{ZFC})$, or a state lower bound. In particular, the theorem is an equivalence; it does not assert unconditional nonhalting.

All predecessor files are taken from ProveIt commit

`75716498437bfc7291d5900cab2983a11fd54bac`.

The two parents are the primary `candidate-k215/` and its `alternatives/commands120/` presentation [2]. Paths to these candidates in the bibliography are relative to `Computability/BusyBeaver/Research/ZFC/`.

Prior work and the actual increment. The Source163 investigation already proposed replacing the formula parameter of ρ_6 by B, with earlier 216/217 sources [5, 6]. It also recorded the same order-210 source hash used here in a source/pricing investigation. We therefore do not claim priority for that symmetry, that source text, or a bare 215-state pricing observation. Here the change is realized as an explicit two-row patch on *each* accepted K215 parent, including the command-120 composition, and is carried through full arbitrary-natural runtime verification and independent blank/source/evidence checks. This is more than a new state numbering; Section 8 gives exact graph comparisons.

2 The exact constructions

2.1 Two transition rows

Apply the following patch to either parent's native table. All other 428 rows are copied byte for byte as transition data.

State	Read	Old action	New action	Effect
199	1	$1\,R\,166$	$1\,R\,12$	choose the B load
206	0	$0\,R\,186$	$0\,R\,76$	corresponding decoder route

No working label is added or deleted. The table syntax is one row `state read write L/R target` per line, together with a `#! start 0` directive. Thus the complete machine is determined by the pinned parent and these two rows; complete tables are also supplied so reconstruction is not required to read the machine.

2.2 The literal source edit

The source is the ordered list of primitive triples `[binary path, opcode, operand]`. Its operations are increment, guarded decrement, break, continue and halt. In the closed sixth-row interval $[888, 1171)$, interchange primitive register operands 3 and 5. Only four occurrences actually change:

$$959, \quad 964, \quad 1137, \quad 1142 \qquad \text{(zero-based primitive ordinals)}.$$

Each changes from register 3 to register 5. All paths, opcodes, other operands and guarded successor ordinals remain identical to the respective parent. There are 4,751 primitives, of which 1,425 are decrements; counting both decrement guards gives 6,176 cases, including the halt case. The nonhalting guarded control graph has 6,175 edges.

At this phase of AX, source register 4 stores A, register 5 stores B, and register 3 stores C. These are phase meanings, not a renaming of the outer logical-register vector. The closed constructor word changes as follows, where lower-case letters load parameters and I, Q, F denote the corresponding numerical constructor operations:

```
old: aacQFIQFIcI
new: aabQFIQFIbI
```

The change replaces two loads of C by loads of B. In native decoder terms, the B and C loads have entries 12 and 166, respectively.

Both sources have 202 distinct hash-consed subtrees, maximum depth 47, and maximum zero run 6. A source-subtree count is not a physical state count; the binary theorem below concerns the actual 215-state tables.

2.3 Source macro transfer on all natural inputs

The source certificate reconstructs the inherited closed parameter-load and constructor trees from the pinned source/table relation. It matches the whole primitive word of the edited interval and independently parses its postfix expression. The register-3/5 swap maps the complete C load macro to the existing B load macro. The intervening constructors touch neither parameter register.

Increment and guarded decrement commute with a bijective renaming of register indices. Break, continue and halt do not inspect register values. Consequently the inherited all-natural macro proof transfers to the renamed load, with the same decreasing loop measures. This argument is about arbitrary natural parameters, not bounded tests of formula codes.

The parameter registers, scratch registers, proof cursor and control continuation are restored exactly. The current formula and theorem stack change precisely by the specified row output. It would be false to claim that the entire register vector is unchanged outside the row: the changed formula code is intentionally retained there. All other AX outputs and stack effects remain as in the parent.

3 A numerical calculus that tolerates junk codes

3.1 Constructors and the zero alias

All constructors in this section are total functions on $\mathbb{N}$:

$$\begin{aligned} P(x,y) &= \frac{(x+y)(x+y+1)}{2} + y, & I(x,y) &= 4P(x,y), \\ R(x,y) &= 4P(x,y)+1, & Q_i x &= 4P(i,x)+2, \\ E(x,y) &= 4P(x,y)+3, & T &= 0, \qquad F = 1, \\ N(x) &= I(x,1), & \mathsf{Ex}_i(x) &= N(Q_i(N(x))). \end{aligned}$$

We write $x \to y$ for $I(x,y)$. The equality $I(0,0) = 0$ is a genuine numerical alias. The proofs never require every natural code to decode as an ordinary first-order formula, and never replace this alias by a free syntactic constructor.

For orientation, the eight retained rows are as follows; the final row is directly enumerated only when $U \le A$:

$$\begin{aligned} \rho_1(A,B) &= \mathsf{Ex}_A(E(A,B)), \\ \rho_2(A,B,C) &= Q_A(B \to C) \to (Q_A B \to Q_A C), \\ \rho_3(A,B,C) &= E(A,B) \to (R(C,A) \to R(C,B)), \\ \rho_4(A,B,C) &= Q_A(Q_B C) \to Q_B(Q_A C), \\ \rho_5(A,B,C) &= E(C,A) \to (R(C,B) \to R(A,B)), \\ \rho_6(A,C) &= \mathsf{Ex}_A(Q_A C) \to C, \\ \rho_7(A,B,C) &= E(C,A) \to (E(C,B) \to Q_{R(A,B)} E(A,B)), \\ \rho_8(A,B,C,U) &= ((C \to U) \to A) \to ((A \to C) \to (B \to C)). \end{aligned}$$

The seventh row is called `growG` in the source contract data. The calculus also contains truth, the exact inherited gated theory bundle, numerical modus ponens and theorem-only generalization. The bundle and its gate are not changed. Its complete numerical constructor tree, postfix word and source spans are retained in `research/formalism/native_axiom_contracts.json`.

3.2 The symmetry that matters for compilation

Lemma 3.1 (Row-local surjective reparameterization)**.** *Suppose a numerical calculus has an axiom family $\{f(p) : p \in D\}$, and replace its enumeration by $\{f(\sigma(q)) : q \in D'\}$, where $\sigma : D' \to D$ is surjective. If the other axiom families and inference rules are unchanged, the set of derivable natural numbers is unchanged.*

Proof. The two sets of axiom codes are equal. Replace an axiom leaf in either finite derivation by a parameter tuple giving the same literal code; all inference nodes and conclusions remain identical. Induction on the derivation gives both inclusions. □

The lemma is local to each axiom family. It does not assume that arbitrary derived numerical codes admit a formula-substitution operation. It also does not require all rows emitted by a single AX command to use one common global parameter renaming.

The original sixth family is $\{\rho_6(a,c) : a, c \in \mathbb{N}\}$; the new family is $\{\rho_6(a,b) : a, b \in \mathbb{N}\}$. They are literally equal sets of natural codes. Thus the old and new native calculi derive exactly the same codes, a stronger property than equiconsistency. The values emitted by a *fixed* AX parameter tuple, however, need not agree. Section 5 repairs the proof compiler accordingly.

The dominated-ρ_8 predecessor theorem recovers every unrestricted ρ_8 instance through a fixed finite derivation, preserving its literal numerical conclusion [4]. Its 3,728-node recovery certificate and native-calculus/ZFC calibration are inherited results. They concern arbitrary numerical instances, including collapsed zero implications. Nothing in our edit changes their premises, the theory bundle or its private numerical codes.

4 Command semantics and soundness

Write a command boundary as (t, S, d, h): current formula, Cantor-coded stack, additive counter, and cached formula. Write $[x]S = P(x, S)$ and extend this notation by nesting to the right. The command contracts are those of the accepted command-permutation compiler [3].

$$\begin{aligned} \mathsf{MP}(z): \quad & b = d + z, && (t, S, d, h) \longmapsto \begin{cases} (b, [0]S, b, 0), & I(t, b) = h, \\ (0, S, b, 0), & I(t, b) \neq h; \end{cases} \\ \mathsf{DROP}: \quad & && (t, S, d, h) \longmapsto (h, [t]S, d, h). \end{aligned}$$

For NIP, first test the *incoming* h. If $h = 1$, halt. Otherwise let $(x, S') = P^{-1}(S)$ and return (t, S', d, x). The test occurs before, not after, the pop.

An AX command has wire fields $[c + 3, y, x]$. On entry, set

$$U = d, \qquad A = d + x, \qquad B = h + y, \qquad C = c.$$

It returns counter/cache zero, current $g(A, B, c)$, and stack

$$[\,\rho_1, \rho_2, \rho_3, \rho_4, \rho_5, \rho_6(A, B), \rho_7, \rho_8, Q_U(h), t\,]S. \tag{1}$$

The other rows use the inherited arguments; ρ_8 uses (A, B, c, U). The gate g is zero if $c = A$ or $c = B$ and otherwise is the exact inherited theory bundle. Since $A = d + x$, the condition $U \leq A$ always holds. In (1), h and t are the incoming cache and current.

The ninth generated entry is a quantifier code, congruent to 2 modulo 4. Every logical row and the bundle is an implication code, divisible by 4. Thus all nine generated entries and the gated current are different from falsity 1, even for junk inputs. There are ten pushes in total, the last saving old current.

The wire tags are:

Presentation	tag 0	tag 1	tag 2
`K215-R6-210`	NIP	DROP	MP
`K215-R6-120`	DROP	NIP	MP

AX tags are $c + 3$ in both. MP consumes its extra numeric field in both.

Lemma 4.1 (Sound command execution)**.** *If t, h, and all successively decoded heads of S are native theorems, every nonhalting command preserves this property. A halt from such a boundary implies that 1 is a native theorem.*

Proof. The changed sixth row is an old allowed axiom instance by Lemma 3.1. AX otherwise uses the unchanged axiom, bundle and generalization rules. MP is precisely the numerical inference test; its failure branch introduces truth 0. DROP and NIP only move theorem codes. NIP's sole halt requires incoming cached theorem 1. The zero stack has only truth heads under repeated unpairing, since $P^{-1}(0) = (0, 0)$. □

5 A repaired completeness compiler

5.1 The strong contract and reset

A strong block for theorem X starts from *any* $(t, S, 0, 0)$ and either halts early or returns $(X, S, 0, 0)$. Current and stack are arbitrary here; counter and cache must be zero. The padding command used later handles fully arbitrary incoming tuples.

The inherited reset is

$$\mathsf{Discard} = \mathsf{MP}(1);\ \mathsf{AX}(1; x = 0, y = 0);\ \mathsf{NIP}^{10}.$$

Against cache 0, $\mathsf{MP}(1)$ fails because $I(t, 1) > 0$. Its result is $(0, S, 1, 0)$. The subsequent AX has $U = A = c = 1$, so its gate is zero. It pushes nine entries different from 1, followed by the saved current 0. Ten NIPs recover the original stack and cache 0. Thus Discard returns $(0, S, 0, 0)$, even from arbitrary t, S.

5.2 Exact row selection

After Discard and an AX call with saved current 0, selecting stack position j, $1 \le j \le 9$, uses

$$\mathsf{NIP}^{j};\ \mathsf{DROP};\ \mathsf{NIP}^{11-j}. \tag{2}$$

Following AX with saved current 0, the first j pops cache the chosen entry. DROP moves it to current and pushes the gated current. The remaining $11 - j$ pops discard that safe gate, the remaining generated rows, and the saved zero. The exact original stack is restored and cache is zero. These identities use arithmetic push/pop laws and remain valid when a push of 0 onto 0 aliases 0.

To produce a desired sixth axiom $\mathsf{Ex}_a(Q_a v) \to v$, the new recipe is

$$\mathsf{Discard};\ \mathsf{AX}(c = 0; x = a, y = v);\ \mathsf{NIP}^{6}; \mathsf{DROP}; \mathsf{NIP}^{5}. \tag{3}$$

The actual emitter has $A = a, B = v, C = 0$; its sixth slot is exactly the target natural code. Other rows emitted by that AX may differ from those of an old compiler call, but they are safely discarded. There is no need to preserve them.

Unchanged rows use their inherited parameter recipes and (2). For ρ_8 with $U \le A$, first set the counter with $J(0) =$ empty and $J(u) = \mathsf{MP}(u)$ for $u > 0$, then use $\mathsf{AX}(c = C; x = A - U, y = B)$ and position 8. The empty $J(0)$ avoids a spurious successful $\mathsf{MP}(0)$ push. An unrestricted ρ_8 leaf is first translated by the inherited dominated-row recovery derivation. Bundle leaves use their original gate witnesses and ten pops; truth uses Discard.

5.3 Inference blocks

The GEN and MP blocks are unchanged. Their proofs depend on stack positions, safe constructor tags and zero gates, not on the internal values of the discarded sixth row. For explicitness, if B_X is a strong block for X, generalization to $Q_i(X)$ uses

$$B_X; \mathsf{DROP}; J(i); \mathsf{NIP}; \mathsf{AX}(i; 0, 0); \mathsf{NIP}^{9}; \mathsf{DROP}; \mathsf{NIP}; \mathsf{NIP}. \tag{4}$$

Before AX the cache is X, counter is i and $U = A = c = i$. The gate is zero and the ninth row is exactly $Q_i(X)$.

Given strong blocks for X and $I(X,Y)$, modus ponens uses

$$B_X;\mathsf{DROP};B_{I(X,Y)};\mathsf{NIP};\mathsf{DROP};\mathsf{NIP};\mathsf{MP}(Y);\\ \mathsf{AX}(Y;0,0);\mathsf{NIP}^{10};\mathsf{DROP};\mathsf{NIP};\mathsf{NIP}. \tag{5}$$

The first line arranges X and $I(X,Y)$ as current and cache for a successful numerical MP. The second resets counter/cache using the zero gate $U = A = c = Y$ and restores the underlying stack. If a pop tests falsity earlier, the required alternative is already halting. Equations (4)–(5) are the inherited all-natural identities, including $X = 0$, $Y = 0$ and collapsed implication aliases [3].

Induction on a finite native derivation, with (3) for its changed axiom leaves, gives a strong block for every native theorem. Only wire encodings of the named NIP/DROP commands change between the 210 and 120 presentations.

5.4 Cofinality and the real initial cutoff

To a strong block for theorem 1 append $\mathsf{DROP};\mathsf{NIP};\mathsf{NIP}$. This either halts earlier or eventually tests cached 1. Prepend the padding command $\mathsf{AX}(0;x=1,y=Y)$. It resets counter/cache from any incoming tuple; the following strong block accepts arbitrary current and stack.

Let $f_\pi(k)$ be the list of numeric fields for command k in wire order π, and let $\mathrm{Pair}(\vec{a},r)$ right-fold those fields using P, with tail r. Explicitly define

$$\mathrm{enc}_\pi([]) = 0, \qquad \mathrm{enc}_\pi(k :: L) = 1 + \mathrm{Pair}(f_\pi(k), \mathrm{enc}_\pi(L)).$$

Let $R = \mathrm{enc}_\pi(B_1;\mathsf{DROP};\mathsf{NIP};\mathsf{NIP})$, where B_1 is the fixed strong refutation block. An initial enumerated candidate has no outer successor, while noninitial command continuations do. With the actual field order, the resulting candidate number is

$$N_Y = P(3, P(Y, P(1, R))) \;\geq Y. \tag{6}$$

This family is cofinal, so a successful refutation candidate occurs above any finite initial cutoff.

The unchanged scheduler increments the next-candidate counter once at each fetch. Positive proof-list pointers strictly drain between commands, field readers return tails no larger than their inputs, and the arithmetic macros terminate on all naturals. Consequently a malformed nonhalting candidate cannot permanently block later candidates [3]. Combined with Lemma 4.1, the source halts from the root

$$[N,T,W,A,D,H,X,Y,L] = [4,0,0,0,0,0,0,0,0] \tag{7}$$

if and only if the native calculus derives 1. The formula-bearing current, cache and stack in (7) are all zero. The cutoff 4 is handled by (6), not by pretending the machine starts enumeration at zero.

6 Whole-tape binary verification

6.1 What is quantified

The unchanged inherited runtime engine is applied to the exact changed table and exact changed primitive list. It proves a binary block for each primitive and each decrement guard. The theorem quantifies over:

- all nine natural primary register values consistent with the guard;
- any natural gap between the program field and register header;
- any nonempty finite spare list of natural register values;
- the complete tape, including the infinite zero exterior.

These are unbounded domains. A finite bound on *symbolic proof construction* is not a bound on natural values or represented runtime.

The register block for value v is $B(v) = 0\,1^{v+1}$. During traversal the alternate form is $C(v) = 1\,0\,1^{v}$. The physical slots for logical indices $0, \ldots, 8$ are

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

The frame contains a left marker, binary program prefix, padding to the header at distance $64+\text{gap}$, nine register blocks and the spare blocks. A logical source step may shift the head/frame origin; the exact affine motion is checked with its successor prefix.

The proof engine uses literal transitions, unary induction for a directional self-loop, and structural induction over a finite list after proving each required one-block schema from the actual table. Its four block schemas traverse B to B rightward, B to C rightward, B to B leftward, and C to B leftward. Each one-block duration is $v + 2$ and preserves arbitrary surrounding contexts.

Every nonhalt primitive returns to the exact successor frame. The halt primitive reaches the distinguished halt state. Each proof has an affine duration in thirteen slots: constant, gap, nine primary values, first-spare value, and the width of the remaining spare list. All coefficients are nonnegative and the constant is strictly positive. Thus each block is finite, and infinitely many source steps require infinitely many binary transitions. This supplies both halting correspondence and divergence preservation.

6.2 Observed complete results

Check	`K215-R6-210`	`K215-R6-120`
Ordered guarded cases	6,176/6,176	6,176/6,176
Symbolic rules	1,699,838	1,699,893
Minimum positive clock constant	6	9
Rows covered, including block schemas	430/430	430/430
Initial recorded runtime time (seconds)	169.69	152.18

Timings describe these executions, not a performance guarantee. All complete case records, the raw progress logs, exact input hashes and startup traces are retained under each machine's directory.

The runtime engine's original legacy command-line driver carries an obsolete source-specific pin. The supplied `code/runtime-inputs/validate_native.py` is the inherited generic native-label wrapper; it applies the unchanged symbolic rules to the current full source. It does not treat an obsolete source pin as a proof of current source semantics. The separate source theorem and source certificate provide that obligation.

6.3 Independent audit and its limits

The standalone `code/audit_candidate.py` imports no supplied checker. It independently parses both tables, resolves the source control graph, checks the exact four edited operands, binds the

runtime report to input hashes, checks all 6,176 cases in order, validates positive clocks and row accounting, checks the four block-row bindings, and verifies all 430 rows of the dense renaming. It then replays the entire blank prefix from an empty sparse tape and builds the complete target frame independently.

This is independent saved-evidence validation and finite replay. It is not a second implementation of the full symbolic induction proof system. Those unbounded proofs were executed by the separately supplied inherited engine. The original observations record successful process completion and report-bound input hashes; they did not record a prelaunch before/after manifest. The optional full reproduction wrapper additionally records before/after hashes of reconstructed machine artifacts and the actual subprocess return status. Neither kind of receipt is a proof-assistant kernel certificate.

7 Exact blank initialization and proof composition

The two complete blank computations reach exactly the same logical frame, at different transition counts:

Property	`K215-R6-210`	`K215-R6-120`
Transitions	95,714	96,284
Native state / head	$84 / -8$	$84 / -8$
Header / gap	$57 / 1$	$57 / 1$
Logical vector	$[4, 0, 0, 0, 0, 0, 0, 0, 0]$	same
Spare list	forty values equal to 1	same
Number of one cells	95	95
Exterior	identically zero	identically zero

The independent replay checks every transition, not just a selected screen or finite tape window. Since the sparse tape starts empty and every write is tracked, its final finite set of one cells also proves the zero exterior. The exact set is in each audit JSON. Neither altered row is visited anywhere in its startup prefix, so the changed machine follows its parent's literal blank prefix.

We can now compose the proof:

1. The exact finite blank run reaches the safe source root (7).
2. The quantified primitive theorem simulates the whole source, with positive durations and exact frame restoration.
3. Soundness, the repaired strong compiler and cofinality show source halting exactly when the numerical calculus derives 1.
4. Row-family equality preserves that derivability condition. The inherited dominated/native-calculus calibration identifies it with the existence of a finite ZFC refutation.

Each implication is arithmetical or an ordinary finite-derivation argument formalizable in ZFC. This proves Theorem 1.1. The binary checker alone would not have supplied steps 1, 3 or 4.

8 Material difference and its precise extent

The auditor compares rooted action-labelled transition graphs. Starting with start mapped to start, input symbol 0 or 1 forces both the action and successor mapping. A forced-map conflict,

injectivity conflict or action mismatch rules out a bijective state renaming. All states are graph reachable, so this deterministic traversal is sufficient.

Both new machines fail the isomorphism test against all four included accepted native references: primary K215, commands120, composed, and composed/source164. They also fail the test against one another. The audit files include the forcing bit word and the conflicting targets for each exclusion. For example, the order-120 machine versus its parent forces a conflict along

```
0000001001000000001000001001110110
```

where the same forced left target 76 would have to match both 76 and 186 on the right. These bit words are paths in the transition graph, not assertions that a physically consistent tape presents that word.

A broader saved corpus comparison for the order-210 machine found no match among sixteen eligible 215-state tables in the checked-out ZFC subtree, even allowing symbol complementation and left/right reflection. This is a bounded corpus statement. It does not establish global historical novelty, semantic inequivalence, or a different blank halting condition. The substantive change is the source's use of a different axiom parameter and its resulting binary routing.

9 Experiments toward 214

The positive construction stops at 215. The following experiments identify finite search boundaries and possible next directions; none is a lower-bound argument.

9.1 All independent row-parameter permutations

For the seven retained ungated rows, assign each of the six permutations of registers 3, 4, 5 independently, preserving the accepted literal source association. The exact search evaluates $6^7 = 279,936$ recipes. The minimum number of distinct source subtrees is 202, attained at exactly four recipes:

Recipe	Row changes	Edited primitive operands
identity	none	0
`joint24`	swap B, C separately in ρ_2, ρ_4	16
`rho6`	use B in ρ_6	4
`joint246`	both preceding changes	20

Every changed family is exactly the original numerical family. For a desired $\rho_j(A, B, C)$ with $j = 2, 4$ in a joint-swap source, the repaired recipe uses $\mathsf{AX}(c = B; x = A, y = C)$ after Discard. The two additional source variants are supplied with their exact source and control-flow certificates; no accepted 215- or 214-state binary allocation is claimed for them.

This census only excludes a source-DAG improvement within the stated finite family and fixed association. It does not exclude another association, a different register program, a cheaper contextual decoder, or a calculus that is merely equiconsistent rather than identical in theorem codes.

9.2 Local transition and source searches

Seven ordinary direct-fold searches evaluated 4,431,512 specifications across the primary, command-120, composed, new rho6 and three rotated seeds. The proof-construction settings were

radius 7 and fuel 97. Additional searches considered 632,584 specifications with a blank initial window, 168,324 and 168,314 shifted-head specifications on two seeds, 94,600 unique-entrance eliminations, and 186,620 first-failure-pruned multiple-entrance probes. No certified reduction survived.

These families overlap; adding their counts does not count distinct machines. Failure of a bounded local proof search can mean that its invariant or proof budget is too weak. It is not a semantic counterexample and does not establish minimality.

A one-row rewrite search found 497 locally stuttering-equivalent proposals among 371,090 alternatives, but no compatible paired fold in the searched closure. Product constant propagation from the calibrated logical seed found only two always-zero decrement sites; the naive source replacement increased the subtree count from 202 to 209. A separate physical-rotation search visited 3,959 distinct tables without dropping below 202 subtrees. These are exploratory receipts; they are not additional accepted machine certificates.

9.3 Controlled global allocation

A corrected global contextual-role allocator fixes 207 inherited engine/helper/context rows and lets source nodes move among existing physical labels. Its source-identity control passes all 4,750 internal prefixes and retains 215 states. That control changes one routing row, so it is not byte-identical to the baseline and has no new full runtime certificate.

Within this fixed-row model, the `joint24` and `joint246` trials are reported infeasible (17,711/17,718 role variables and 15,503/15,514 constraints). A broader model fixes only 188 rows and gives nineteen contextual entries a keep-or-reassign decision. Its `joint24` trial reached its 60-second solver limit, measured as 63.05 seconds, with no feasible incumbent. This is a timeout, not an infeasibility result. Earlier draft models that rejected the identity control are excluded from these negative claims.

The portable allocation scripts, inputs, exact solver records and attribution are included under `research/independent-encoding/`. This family still restricts the machine architecture, so its failures do not address general 215- or 214-state realizability.

9.4 A useful rejected command permutation

Changing primary K215 to NIP0/MP1/DROP2 by

$$134/1:\ 1R12 \mapsto 1R135, \qquad 174/1:\ 1R135 \mapsto 1R12$$

retains 215 states and passes all source-prefix checks. An observed symbolic run also completed all 6,176 primitive cases. Nevertheless the literal machine *halts from blank after 646,553 transitions.*

Its first complete source root occurs at transition 494,760 with

$$[N, T, W, A, D, H, X, Y, L] = [2, 0, 4, 0, 1, 0, 0, 0, 0].$$

The stack value $4 = P(1, 1)$ has falsity as its first head. Candidate $2 = P(0, 1)$ encodes NIP;NIP under this tag order. The first NIP loads the bad head and the second tests it. The finite rejection is independently reproducible with a standard-library simulator; the final tape contains exactly 409 one cells and zero exterior. The rejection does not depend on the symbolic-run observation, whose full per-case records are not retained in this supplement.

This example pinpoints why a correct primitive interpreter and an equiconsistent source language do not alone prove the desired blank-machine theorem. The initial theorem-stack invariant is also essential.

9.5 Why dropping a shared PUSH is not yet a formalism

Bypassing native state 164's shared PUSH role would omit four AX pushes as well as its other use. The load helper adds its parameter to the uncleared current; it does not overwrite that current. Consequently some leading quantifier indices become A plus the preceding row code, while other occurrences of A remain unchanged. For example, the sixth row becomes

$$\mathsf{Ex}_{A+G}(Q_A C) \to C,$$

where G is the preceding completed row code, rather than a uniform reparameterization of ρ_6. AX would also lose four stack outputs. There is no soundness/equiconsistency proof for that machine here. The exact symbolic obstruction is retained in `research/formalism/shared_push_obstruction.md`.

The next promising boundary. The symmetry census suggests searching the *presentation* of axiom families jointly with decoder context: permit row-local surjections, then compile an explicit right-inverse selection recipe. The two joint variants show that the logical freedom is larger than the single ρ_6 change. To reach 214, an allocator likely needs to free an additional contextual role or change source association, or a new formalism needs a raw-code calibration robust to a more substantial stack/constructor change. These are research directions, not completed claims.

10 Reproduction, review and integration

10.1 One-command positive reproduction

Python 3.10+ with assertions enabled suffices for all positive checks. No solver, network connection, external repository checkout or third-party Python package is required. From the unpacked packet root:

```
python3 -B code/reproduce.py --output fresh-audit
python3 -B code/reproduce.py --full-runtime --output fresh-runtime
```

Output directories must not exist. The first command reconstructs both exact tables and sources from the included parents, checks expected hashes, runs the symbolic source certificate, and audits the archived full runtime evidence and blank traces. The second additionally executes the full inherited symbolic runtime checker on both reconstructed candidates and audits the new results. Allow several minutes; use `-timeout` to increase the per-runtime timeout on a slower system. Do not use Python's `-O` option or set `PYTHONOPTIMIZE`.

The finite row census can be rerun separately:

```
python3 -B code/row_combo_search.py \
  --seed baseline/candidate-k215 --output fresh-row-census
```

The rejected blank machine uses `research/independent-encoding/negative/reproduce.py`. Its allocator supplements additionally require NumPy and SciPy's MILP interface; they are not dependencies of the positive result.

10.2 Packet map

Location	Purpose
`machines/rho6_120/`	Recommended candidate: native/dense tables, source, mapping, full runtime cases, startup trace, log and receipt.
`machines/rho6_210/`	Companion candidate with the primary command order.
`baseline/`	Pinned parents, four graph-comparison references and inherited source readers.
`code/`	Deterministic reconstruction, source certificate, runtime engine and independent auditor.
`evidence/`	Independent audits, graph comparison, source certificate and portable reproduction records.
`research/`	Additional calibrated source variants, search receipts, global allocation supplement and rejected-machine replay.
`references/`, `licenses/`	Inherited proof expositions, original notices and license terms.
`manifest.json`, `SHA256SUMS.json`	Exact construction/input pins and final archive-file checksums.
`report.tex`, `report.pdf`	This manuscript in editable and rendered form.

For incorporation, the recommended machine directory can become a new K215 alternative, with the companion retained for comparison. Dense labels enumerate the surviving native working labels in increasing order; use the included exact bijection for each presentation. Do not combine a table from one command order with the other order's source. Original records retain absolute working paths as historical provenance; the supplied positive reproduction scripts use paths relative to their own packet and do not depend on those locations.

10.3 Review priorities

The most important mathematical review is the source-family lemma together with the repaired selector (3) and the unchanged native calibration. The most important implementation review is the two-row patch, the four source operands, quantified runtime, and complete safe startup. Search failures can be reviewed independently; they are not premises of the positive theorem.

Original attribution is preserved. The inherited notices credit ProveIt and its submitted constructions, Elliot Glazer's compact theory K/K230 work, Jeff's busy-beaver/Rocq reference, Andrew James Wade's compiler ancestry, BusyCoq, and the Coq Library of Undecidability within their respective scopes. New material is provided under the repository's MIT No Attribution terms without relicensing inherited components. Reviews in this packet are separate agent reviews inside the research collaboration, not an external human endorsement.

A Exact candidate identities

SHA-256 values below bind the machine/source bytes. The full manifest also pins the parent inputs, canonicalization records, runtime data and verification implementation.

`K215-R6-210`. Native table

`9c2f0502ec50718452e652a56e49ed77409e3edcd8e1643a7f6db9e2acadc494`

Dense table

`87a9989583b80b99ab6b31f667e19595f9aa594dfcb89441285fefef7c32f9da`

Source

`19da13e482ea0f3d57432aad852bdf2448093e58a6e3b3873ac744eae7440693`

`K215-R6-120`. Native table

`0d5ea012bceed31783e39b9f13dba3e76ba656ed7ee1fb2f2106d2417faeab2f`

Dense table

`8d1b8b49996ce96d814092f44d03e5ecca9ef59e2014c06b84506f2b5c98a899`

Source

`da5b11914de489e25007ec89b8dd5109d8f234da2abc9b552e6f7b31e9957f11`

References

[1] ProveIt, *Combined K215 candidate and ordinary proof*, pinned commit `75716498437bfc7291d5900cab2983a11fd54bac`. candidate-k215/proof.md. The parent source, table and exposition are included under `baseline/candidate-k215/`.

[2] ProveIt, *K215 command-order 120 alternative*, same commit. candidate-k215/alternatives/commands120/README.md.

[3] ProveIt, *Ordinary proof for the command-permutation alternatives*, same commit. candidate-k216/alternatives/permuted/proof.md. Local copy: `references/command-permutation-proof.md`.

[4] ProveIt, *Ordinary proof boundary of the dominated-rho8 machine*, Source149, same commit. candidate-k218/alternatives/dominated/proof.md. Local copy: `references/dominated-rho8-proof.md`.

[5] ProveIt, *Rho6 parameter-permutation alternative*, Source163, same commit. candidate-k216/alternatives/rho6/README.md. Local copy: `references/prior-rho6-README.md`.

[6] ProveIt, *Intake 199 / Source163 dossier and price investigation*, same commit. intake199-source163-dossier.md. This predecessor is distinguished from the new full binary checks in the present packet.